Javascript

Prof. Rafael Vinicius Germinari Gonçalves
Bacharel em Ciência da Computação - UFMS

Executando js no navegador

<script src="script.js"></script>: Este é um elemento **<script>** que é usado para incorporar código JavaScript na página. A parte importante aqui é o atributo **src** (source), que aponta para um arquivo externo chamado "script.js". Isso significa que o código JavaScript a ser executado está contido no arquivo "script.js", que deve estar no mesmo diretório que o arquivo HTML ou em um caminho acessível a partir dele.

Cuidado:
Ao colocar o elemento <script> dentro do <body> desta maneira, você garante que o código JavaScript será executado após o carregamento da página. Se o <script> estivesse no <head>, o navegador tentaria executá-lo antes de carregar o conteúdo da página, o que poderia causar problemas se o script dependesse de elementos HTML que ainda não foram carregados.

```
<body>
    <script src="script.js"></script>
</body>
```

Variáveis

Uma variável em JavaScript é um local de armazenamento que contém dados. Para criar uma variável, usamos a palavra-chave **var**, **let**, ou **const**.

- **var :** possui escopo global dentro do bloco.
- **let :** permite declarar variáveis com escopo de bloco. Recomendado para a maioria dos casos de declaração de variáveis.
- **const:** declara uma constante com escopo de bloco. O valor de uma constante não pode ser reatribuído.

```
var idade = 25;
let nome = "João";
const PI = 3.14;
```

JavaScript é uma linguagem dinamicamente tipada, o que significa que o tipo de uma variável é determinado durante a execução do programa.

Tipos primitivos

- String: Sequência de caracteres.
- Number: Números inteiros ou de ponto flutuante.
- Boolean: Representa verdadeiro ou falso.
- Null: Valor nulo intencionalmente.
- Undefined: Variável que foi declarada, mas não foi atribuída com um valor.

```
let nome = "amanda";
let idade = 22;
let casada = true;
let salario = null;
let endereco;

console.log("nome:", nome, typeof nome);
console.log("idade :", idade , typeof idade );
console.log("casada :", casada , typeof casada );
console.log("salario :", salario , typeof salario );
console.log("endereco:", endereco, typeof endereco);
```

Strings

Declarando uma String: Use aspas simples (') ou duplas (") para criar strings.

Concatenação de Strings: Combine strings com o operador de adição (+).

Comprimento da String: Obtenha o número de caracteres usando **length**.

Acessando Caracteres: Acesse caracteres por índices (iniciando em zero).

Template Strings: Utilize crases (``) para criar strings com interpolação de variáveis.

Métodos de String: Além das operações básicas, alguns métodos úteis incluem:

- **toUpperCase():** Transforma todos os caracteres em maiúsculas.
- **toLowerCase():** Converte todos os caracteres em minúsculas.
- **indexOf():** Retorna o índice da primeira ocorrência de uma substring.
- **substring():** Extrai uma parte da string com base em índices.

```
let firstName = 'John';
let lastName = 'Doe';
let fullName = firstName + ' ' + lastName;

let message = 'Hello, World!';
let length = message.length;

let greeting = 'Hello';
let firstChar = greeting[0];

let name = 'Alice';
let greetingMessage = `Hello, ${name}!`;

let text = 'JavaScript is powerful';
let upperCaseText = text.toUpperCase();
let index = text.indexOf('is');
let subString = text.substring(0, 10);
```

Number

Declarando Variáveis Numéricas: Você pode declarar uma variável numérica usando a palavra-chave **var**, **let**, ou **const**.

Operações Numéricas: JavaScript suporta várias operações numéricas padrão, como adição, subtração, multiplicação e divisão.

toFixed(digits): Este método retorna uma string representando o número com um número especificado de casas decimais.

toPrecision(precision): retorna uma string representando o número com uma precisão específica (número total de dígitos significativos).

toString([base]): Este método converte o número para uma string. A base opcional pode ser especificada para converter o número para uma representação em outra base (por exemplo, binário, octal, hexadecimal).

```
var inteiro = 42;
var pontoFlutuante = 3.14;

var soma = 5 + 3;
var subtracao = 10 - 4;
var multiplicacao = 2 * 6;
var divisao = 8 / 2;

var numero = 123.456789;
console.log(numero.toFixed(2));
console.log(numero.toPrecision(4));
console.log(numero.toString());
```

Math

Math.sqrt(x): Retorna a raiz quadrada de um número.

Math.abs(x): Retorna o valor absoluto de um número, ou seja, a distância desse número até zero.

Math.round(x): Arredonda um número para o inteiro mais próximo.

Math.floor(x): Arredonda um número para baixo, para o inteiro mais próximo.

Math.ceil(x): Arredonda um número para cima, para o inteiro mais próximo.

Math.pow(base, expoente): Retorna a base elevada à potência do expoente.

Math.min(...args): Retorna o menor valor entre os argumentos fornecidos.

Math.max(...args): Retorna o maior valor entre os argumentos fornecidos.

Math.random(): Retorna um número pseudoaleatório no intervalo de 0 (inclusive) a 1 (exclusivo).

```
var raizQuadrada = Math.sqrt(9);
var absoluto = Math.abs(-5);
var arredondado = Math.round(4.2);
var paraBaixo = Math.floor(4.9);
var paraCima = Math.ceil(4.2);
var potencia = Math.pow(2, 3);
var menorValor = Math.min(4, 5, 3);
var maiorValor = Math.max(4, 5, 3);
var aleatorio = Math.random();
```

Dates

JavaScript possui um objeto integrado chamado Date para lidar com datas e horas. Para criar uma nova instância representando a data e hora atuais, use **new Date()**. Da mesma forma, você pode criar uma instância a partir de uma string de data específica, como exemplificado por **new Date('12-07-2023 12:00:00')**.

Métodos para obter componentes de data e hora:

- **getFullYear():** Obtém o ano da data.
- **getMonth():** Obtém o mês da data (0 a 11).
- **getDate():** Obtém o dia do mês.
- **getHours():** Obtém a hora do dia.
- **getMinutes():** Obtém os minutos da hora.
- **getSeconds():** Obtém os segundos do minuto.

Métodos para formatar datas:

- **toLocaleString():** Retorna uma representação da data e hora formatada de acordo com as configurações locais do ambiente.

- **toLocaleDateString("pt-BR"):** Retorna a representação da data formatada, excluindo as informações de hora e seguindo o formato brasileiro.

```
let x = new Date();
z = new Date('12-07-2023');

y = x.getFullYear();
y = x.getMonth();
y = x.getDate();
y = x.getHours();
y = x.getMinutes();
y = x.getSeconds();

x.toLocaleString()
x.toLocaleDateString("pt-BR")
```

Criar formatação

Formato de Data:

- '**numeric**': Representação numérica (por exemplo, 2, 05, 2019).
- '**2-digit**': Representação numérica com dois dígitos (por exemplo, 02, 05, 19).
- '**long**': Nome completo (por exemplo, "dezembro").
- '**short**': Nome abreviado (por exemplo, "dez").

Formato de Hora:

- '**numeric**': Representação numérica (por exemplo, 2, 05).
- '**2-digit**': Representação numérica com dois dígitos (por exemplo, 02, 05).
- '**long**': Nome completo (por exemplo, "horas").
- '**short**': Nome abreviado (por exemplo, "h").

Formato de Dia da Semana(weekday):

- '**long**': Nome completo (por exemplo, "quinta-feira").
- '**short**': Nome abreviado (por exemplo, "qui").
- '**narrow**': Nome estreito (por exemplo, "Q").

Formato de Ano:

- '**numeric**': Representação numérica completa (por exemplo, 2019).
- '**2-digit**': Representação numérica com dois dígitos (por exemplo, 19).

```
let options = {
    weekday: 'long',
    year: '2-digit',
    month: 'long',
    day: 'numeric',
    hour: 'numeric',
    minute: 'numeric',
    second: 'numeric',
}
console.log(x.toLocaleString('pt-BR', options))
```

Operadores

Aritméticos

- **+** (adição)
- **-** (subtração)
- ***** (multiplicação)
- **/** (divisão)
- **%** (módulo, retorna o resto da divisão)

Atribuição

- **=** (atribuição)
- **++,+=, -=, *=, /=** (atribuição com operação)

Comparação

- **==** (igualdade, com coerção de tipo)
- **===** (igualdade estrita, sem coerção de tipo)
- **!=** (diferença, com coerção de tipo)
- **!==** (diferença estrita, sem coerção de tipo)
- **>, <, >=, <=** (maior que, menor que, maior ou igual, menor ou igual)

Lógicos

- **&&** (E lógico)
- **||** (OU lógico)
- **!** (NÃO lógico)

Estruturas de controle de fluxo

As estruturas de controle de fluxo em JavaScript, tais como **if, else, switch, for, while e do-while,** desempenham um papel fundamental na gestão do fluxo de execução do código com base em condições específicas.

- O **if** representa a estrutura condicional mais básica, possibilitando a execução de um bloco de código somente se uma condição especificada for avaliada como verdadeira.

- Quando utilizado em conjunto com o **if**, o **else** especifica um bloco de código a ser executado caso a condição do **if** não seja verdadeira.

- O **switch** é empregado quando há a necessidade de avaliar uma expressão em relação a múltiplos valores possíveis, proporcionando uma alternativa mais clara e eficiente em comparação com múltiplos **ifs** encadeados.

- O loop **for** é particularmente útil quando se tem conhecimento prévio da quantidade exata de iterações desejadas sobre um bloco de código.

- O loop **while**, por sua vez, é apropriado quando se deseja repetir um bloco de código enquanto uma condição específica permanecer verdadeira.

- O loop **do-while** assemelha-se ao **while,** porém, garante a execução do bloco de código pelo menos uma vez, mesmo que a condição seja falsa inicialmente. Esta característica pode ser valiosa em determinados cenários de implementação.

Estruturas de controle de fluxo

IF ELSE EXEMPLO

Neste exemplo, temos uma variável **nota** que armazena a pontuação de um aluno em um exame (em uma escala de 0 a 100).

Essa abordagem com **else if** é útil quando você tem várias condições mutuamente exclusivas, e você só deseja que uma delas seja executada com base nas circunstâncias. O uso de else garante que apenas um bloco seja executado, mesmo que várias condições sejam verdadeiras.

```
let nota = 75;

if (nota >= 90) {
    console.log("A");
} else if (nota >= 80) {
    console.log("B");
} else if (nota >= 70) {
    console.log("C");
} else {
    console.log("Nota abaixo de C");
}
```

Estruturas de controle de fluxo

SWITCH EXEMPLO

Neste exemplo, **switch** é utilizado para associar um nome ao dia da semana com base no valor de **diaSemana.**
Cada **case** representa um valor possível da expressão.
O código dentro de cada case é executado até encontrar a instrução **break**. O **default** é opcional e é executado se nenhum dos case corresponder ao valor da expressão.

```
let diaSemana = 3;
let nomeDia;

switch (diaSemana) {
    case 1:
        nomeDia = "Domingo";
        break;
    case 2:
        nomeDia = "Segunda-feira";
        break;
    case 3:
        nomeDia = "Terça-feira";
        break;
    // ... outros casos ...
    default:
        nomeDia = "Dia desconhecido";
}
```

Estruturas de controle de fluxo

FOR EXEMPLO

Neste exemplo, o loop “**for**” começa com “**i**” igual a 0, executa o bloco de código enquanto “**i**” for menor que 5 e incrementa “**i**” a cada iteração.

```
for (let i = 0; i < 5; i++) {
    console.log(i);
}
```

Estruturas de controle de fluxo

WHILE EXEMPLO

Neste exemplo, o bloco de código dentro do loop **while** é executado enquanto “**i**” for menor que 5.

```
let i = 0;
while (i < 5) {
    console.log(i);
    i++;
}
```

Estruturas de controle de fluxo

DO-WHILE EXEMPLO

Neste exemplo, o bloco de código dentro do loop **do-while** é executado pelo menos uma vez, pois a condição é verificada após a execução do bloco.

```
let i = 0;
do {
    console.log(i);
    i++;
} while (i < 5);
```

Funções

NORMAL

```
function soma(a, b) {
  return a + b;
}

let resultado = soma(2, 3);
```

As funções normais podem ser declaradas e usadas em qualquer lugar no código, e o nome da função é obrigatório.

ANÔNIMA

```
let soma = function(a, b) {
  return a + b;
};

let resultado = soma(2, 3);
```

As funções anônimas não têm um nome associado diretamente à declaração. Elas são frequentemente utilizadas em situações em que a função é passada como argumento para outras funções (como em callbacks)

ARROW

```
let soma = (a, b) => {
  return a + b;
};

let resultado = soma(2, 3);
```

Arrow functions oferecem uma sintaxe mais curta, o que pode tornar o código mais limpo, principalmente para funções simples.

Estruturas de Dados

Array ou vetores:

Um array em JavaScript é um objeto que permite armazenar vários valores em uma única variável. Os elementos de um array podem ser de diferentes tipos e podem ser acessados por meio de um índice.

```
// Forma literal
let meuArray = [valor1, valor2, valor3];

// Usando o construtor Array
let outroArray = new Array(valor1, valor2, valor3);
```

A forma mais comum de criar um array em JavaScript é usando a sintaxe literal. Você pode definir um array literalmente, especificando os elementos entre colchetes [].

Estruturas de Dados

ARRAY EXEMPLO

Os elementos de um array são acessados através de um índice baseado em zero.

Propriedade length:
- Retorna o número de elementos em um array.

Método push(elemento1, ..., elementoN):
- Adiciona elementos ao final do array.

Método pop():
- Remove o último elemento do array.

Método shift():
- Remove o primeiro elemento do array.

Método unshift():
- Adiciona elementos ao início do array.

Método splice(inicio, fim):
- Permite fazer uma copia do conteúdo de um array, entre o inicio e fim mas o fim não está incluso.

```
let frutas = ['Maçã', 'Banana', 'Morango'];

console.log(frutas[0]); // Saída: Maçã

console.log(frutas.length); // Saída: 3

frutas.push('Pera');

let ultimaFruta = frutas.pop();

let primeiraFruta = frutas.shift();

frutas.unshift('Uva', 'Abacaxi');

// Remove 1 elemento a partir do índice 1
frutas.splice(1, 1);

// Adiciona 'Melancia' a partir do índice 2
frutas.splice(2, 0, 'Melancia');
```

Estruturas de Dados

ARRAY LOOP EXEMPLO

Pode-se usar loops (como **for)** para percorrer os elementos do array.

```
for (let i = 0; i < frutas.length; i++) {
  console.log(frutas[i]);
}
```

Loop avançado

FOREACH EXEMPLO

O método **forEach** é um método de iteração disponível em arrays em JavaScript. Ele permite executar uma função de callback para cada elemento do array, proporcionando uma maneira concisa e legível de percorrer os elementos sem a necessidade de usar um loop **for**.

Sintaxe:
array.forEach(function(elemento, indice) { // Código a ser executado para cada elemento });

- **elemento:** O valor do elemento atual do array durante a iteração.
- **indice:** O índice do elemento atual no array.
- **arrayCompleto:** O array ao qual o elemento pertence.

```
let frutas = ['Maçã', 'Banana', 'Morango'];

frutas.forEach(function(fruta, indice) {
  console.log(`Índice ${indice}: ${fruta}`);
});
```

Estruturas de Dados

Objetos:

Em JavaScript, um objeto é uma estrutura de dados que armazena dados e funcionalidades relacionadas

```
let pessoa = {
  nome: "João",
  idade: 25,
  profissao: "Desenvolvedor"
};
```

Estruturas de Dados

OBJETOS EXEMPLO

Acesso às Propriedades: Para acessar as propriedades de um objeto em JavaScript, você pode utilizar duas formas: através do operador de ponto (**pessoa.nomeDaPropriedade**) ou do operador de colchetes (**pessoa["nomeDaPropriedade"]**). Ambas as abordagens proporcionam acesso aos valores associados às propriedades do objeto.

Adição e Atualização de Propriedades: Para adicionar ou atualizar propriedades em um objeto, é possível realizar a operação de atribuição. Ao atribuir um novo valor a uma propriedade existente, você efetivamente substitui o valor anterior, proporcionando uma maneira simples e direta de modificar dados dentro do objeto.

Métodos: Além de armazenar dados, objetos em JavaScript podem conter métodos, que são funções associadas a eles. A inclusão de métodos permite a definição de comportamentos específicos que podem ser invocados quando necessário. Essa flexibilidade na criação de métodos dentro dos objetos contribui para a construção de estruturas mais dinâmicas e funcionais.

```
let pessoa = {
  nome: "João",
  idade: 25,
  profissao: "Desenvolvedor"
};

console.log(pessoa.nome); // Saída: João
console.log(pessoa["idade"]); // Saída: 25

pessoa.endereco = "Rua ABC, 123";
pessoa.idade = 26;

let pessoaNova = {
  nome: "João",
  idade: 25,
  apresentar: function() {
    console.log(`Olá, eu sou ${this.nome} e tenho ${this.idade} anos.`);
  }
};

pessoaNova.apresentar(); // Saída: Olá, eu sou João e tenho 25 anos.
```

JSON (JavaScript Object Notation):

O JSON é um formato de intercâmbio de dados leve e fácil de ler. Ele é usado para representar dados estruturados como pares chave-valor e listas ordenadas. Embora tenha "JavaScript" no nome, o JSON é independente de linguagem e é amplamente utilizado em diferentes contextos, especialmente na comunicação entre sistemas web.

Características do JSON:

1. **Sintaxe Simples:**
   - O JSON usa uma sintaxe simples para representar dados, tornando-o fácil de ler e escrever tanto para humanos quanto para máquinas.
2. **Estrutura de Dados:**
   - Os dados em JSON são organizados em pares chave-valor, semelhante à forma como os objetos são estruturados em JavaScript.
3. **Tipos de Dados Suportados:**
   - JSON suporta strings, números, objetos, arrays, booleanos e valores nulos. Isso o torna versátil para representar uma variedade de dados.

```
let pessoa = {
  nome: "João",
  idade: 25,
  profissao: "Desenvolvedor"
};

let pessoaJSON = JSON.stringify(pessoa);
console.log(pessoaJSON);
```

```
let pessoaJSON = '{"nome":"João","idade":25,"profissao":"Desenvolvedor"}';
let pessoa = JSON.parse(pessoaJSON);
console.log(pessoa);
```

Convertendo JSON para objeto do javascript

Aplicações do JSON:

- **Comunicação entre Servidor e Cliente:**
  - É comum usar JSON para transmitir dados entre um servidor e um cliente em aplicativos web.

- **Armazenamento de Configurações:**
  - Arquivos de configuração frequentemente usam o formato JSON devido à sua legibilidade e facilidade de análise.

- **Persistência de Dados:**
  - O JSON é usado em muitos sistemas para armazenar e recuperar dados de forma estruturada.

- **Configuração de APIs:**
  - Muitas APIs retornam ou aceitam dados no formato JSON.

Com essa compreensão, os desenvolvedores podem aproveitar o JSON para várias finalidades, desde a troca de dados em aplicativos web até o armazenamento de configurações e persistência de dados.

DOM(Document Object Model)

é uma interface de programação que representa documentos HTML ou XML como uma árvore de objetos. Cada parte de um documento, como tags, atributos e conteúdos de texto, é modelada como um objeto que pode ser manipulado por linguagens de programação, como JavaScript.

- Manipulação dinâmica: Através do DOM, você pode acessar e modificar a estrutura, estilo e conteúdo de um documento web de forma dinâmica. Por exemplo, é possível:
- Alterar o texto de um parágrafo.
- Adicionar ou remover elementos HTML.
- Mudar estilos CSS de forma programática.

- Eventos: O DOM permite associar eventos, como cliques ou toques, a elementos. Com isso, você pode criar interações dinâmicas e respostas a ações do usuário.

Acesso aos Elementos

Para manipular o DOM, primeiro você precisa acessar os elementos da página. JavaScript oferece métodos para selecionar esses elementos:

document.getElementById(): Seleciona um elemento pelo seu ID.
document.getElementsByClassName(): Seleciona todos os elementos com uma determinada classe.
document.getElementsByTagName(): Seleciona todos os elementos de uma determinada tag.
document.querySelector(): Seleciona o primeiro elemento que corresponde a um seletor CSS.
document.querySelectorAll(): Seleciona todos os elementos que correspondem a um seletor CSS.

```
1
2  const elemento = document.getElementById("meuId");
3  const elementos = document.getElementsByClassName("minhaClasse");
4  const paragrafos = document.getElementsByTagName("p");
5  const primeiroElemento = document.querySelector(".classe");
6  const todosElementos = document.querySelectorAll("div.classe");
7
```

getElementById

Elemento HTML:

- Quando o método encontra um elemento com o ID especificado, ele retorna um objeto representando esse elemento no DOM. Esse objeto possui propriedades e métodos que podem ser usados para manipular o elemento.
- O tipo de retorno é um objeto HTML. Por exemplo, se você tiver um , o retorno será um objeto do tipo HTMLDivElement.

HTML

```
1  <div class="card" id="cardID"></div>
```

Console.log

```
<div class="card" id="cardID"></div>
```

getElementsByClassName

Retorno como HTMLCollection
O getElementsByClassName() retorna um objeto do tipo HTMLCollection, que é uma coleção dinâmica de elementos. Isso significa que, se o documento HTML for modificado (por exemplo, um elemento for adicionado ou removido), o HTMLCollection será atualizado automaticamente.

Esse retorno não é um array, mas sim uma coleção de elementos que pode ser acessada de maneira semelhante a um array, através de índices numéricos.

HTML

```
1  <div class="card"></div>
2  <div class="card"></div>
3  <div class="card"></div>
```

Console.log

```
▼ HTMLCollection(3) [div.card, div.card, div.card]
  ▶ 0: div.card
  ▶ 1: div.card
  ▶ 2: div.card
    length: 3
  ▶ [[Prototype]]: HTMLCollection
```

Coisas importantes sobre getElementsByClassName

. Propriedades da HTMLCollection

- Dinamismo: Como mencionado, a HTMLCollection se atualiza automaticamente. Se elementos forem adicionados ou removidos da página, a coleção refletirá essas mudanças em tempo real.

- **Não é um array real: Embora a HTMLCollection possa ser percorrida com um loop for, não possui todos os métodos de um array (como forEach(), map(), etc.). Se precisar dessas funcionalidades, você pode converter a HTMLCollection em um array:**
  - **const elementosArray = Array.from(elementos);**

. Vários Elementos ou Nenhum

O getElementsByClassName() sempre retorna uma coleção:

- Se houver elementos com a classe especificada, você obterá uma coleção contendo esses elementos.
- Se não houver elementos com a classe especificada, você receberá uma coleção vazia, mas ainda será uma HTMLCollection com tamanho 0.

```
1  //Acessar elementos pelo índice:
2  const primeiroElemento = elementos[0];
3  //Verificar o tamanho da coleção:
4  const totalElementos = elementos.length;
5  //Percorre a coleção
6  for(let i = 0; i < elementos.length;i++){
7      console.log(elementos[i])
8  }
```

querySelector

O método querySelector() em JavaScript retorna o primeiro elemento do DOM que corresponde a um seletor CSS especificado. Se não houver nenhum elemento correspondente, ele retorna null.

Características principais do querySelector():
Seleção única: Ele retorna apenas o primeiro elemento encontrado que corresponde ao seletor fornecido, mesmo que existam vários elementos correspondentes na página. Para selecionar todos os elementos que correspondem ao seletor, utiliza-se o método **querySelectorAll().**

Seletores CSS: O querySelector() aceita qualquer seletor válido de CSS, como:

- Classes (com o . antes do nome da classe).
- IDs (com o # antes do nome do ID).
- Tags (sem prefixo).
- Combinações de seletores (por exemplo, "div.container" ou "ul li a").

HTML

```html
1  <div class="card"></div>
2  <div class="card"></div>
3  <div class="card"></div>
```

querySelector

```
▶ div.card
```

querySelectorAll

```
▼ NodeList(3) [div.card, div.card, div.card]
  ▶ 0: div.card
  ▶ 1: div.card
  ▶ 2: div.card
    length: 3
  ▶ [[Prototype]]: NodeList
```

Modificação de Conteúdo

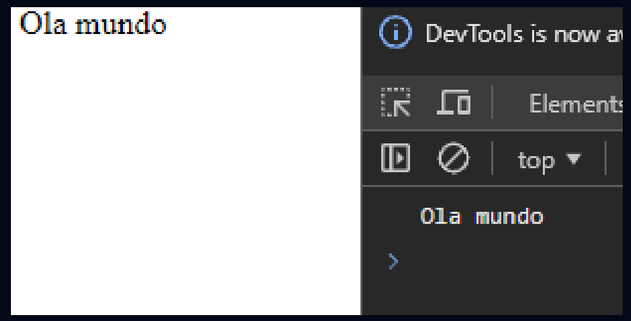
Após acessar um elemento, você pode alterar seu conteúdo ou propriedades.

```html
<div id="cardID"></div>
```

Html

```javascript
const elemento = document.getElementById("cardID");
console.log(elemento.innerHTML = "Ola mundo")
```

Javascript



Resultado

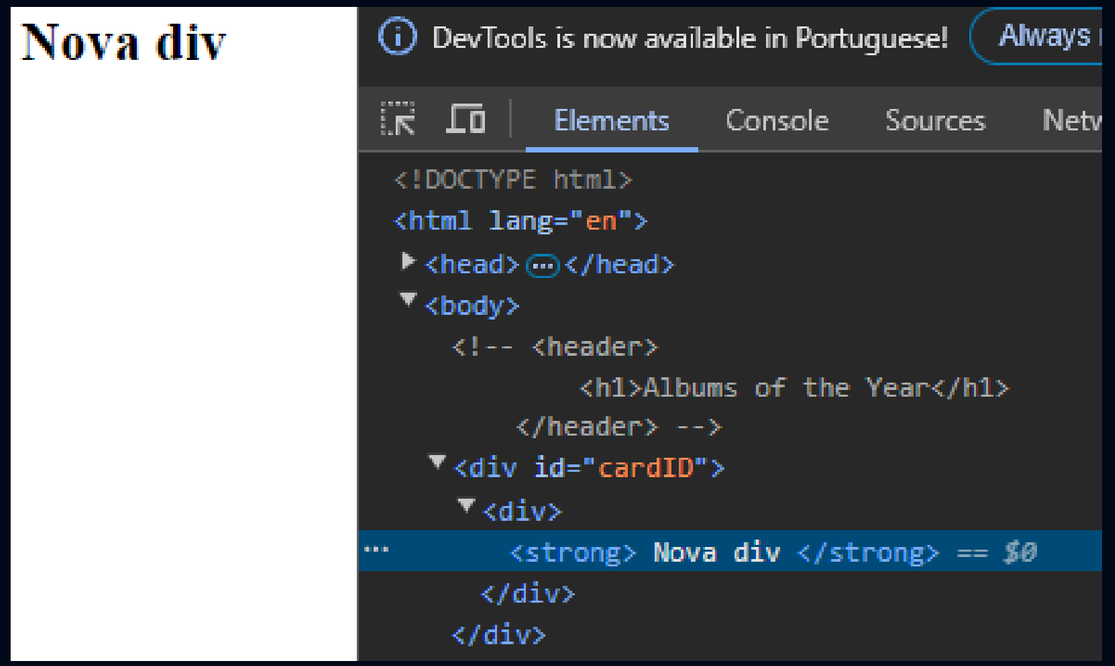
Alterar o HTML interno

É possível inserir código HTML usando a propriedade innerHTML, permitindo que a página renderize esse novo conteúdo dinâmico

Javascript

```
1  const elemento = document.getElementById("cardID");
2  elemento.innerHTML = "<div> <strong> Nova div </strong> </div>"
3  console.log(elemento)
```

Resultado



Criação e Inserção de Elementos

A criação e inserção de elementos no DOM são operações fundamentais para a construção dinâmica de páginas web. Isso permite que você adicione novos elementos a uma página em resposta a ações do usuário ou a outras condições. Vamos explorar esses conceitos em mais detalhes

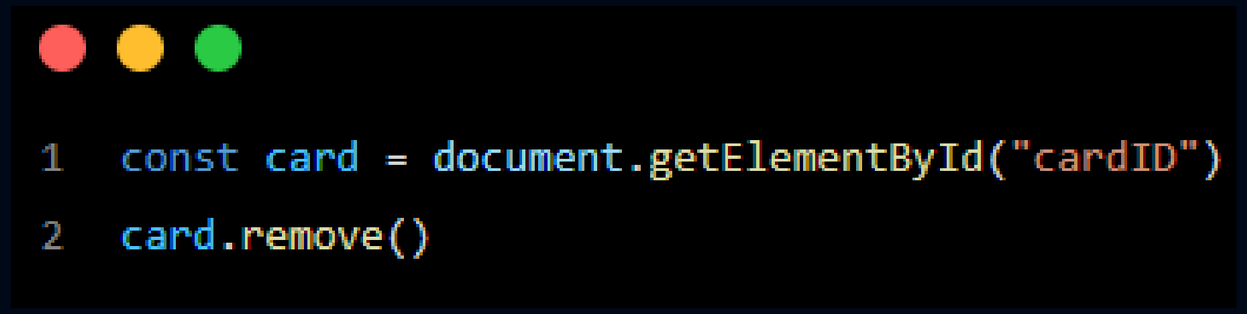
Para criar um novo elemento, você utiliza o método **document.createElement()**. Esse método aceita como argumento o nome da tag que você deseja criar.

Depois de criar um novo elemento, o próximo passo é inseri-lo em um lugar específico da página. Para isso, você pode usar métodos como **appendChild()**

```
1  const novoElemento = document.createElement("div");
2  novoElemento.innerText = "Este é um novo elemento!";
3  novoElemento.setAttribute("class", "meuElemento");
4
5  const container = document.getElementById("container");
6  container.appendChild(novoElemento);
```

Remoção de Elementos

O método remove() é o mais direto e simples para remover um elemento do DOM. Quando você chama **remove()** em um nó, ele é excluído da árvore do documento



Navegação entre Elementos

Navegar entre os elementos do DOM é um conceito fundamental que permite acessar e manipular diferentes nós na estrutura hierárquica do documento HTML. A seguir, estão os principais métodos e propriedades que você pode usar para navegar entre os elementos:

- parentNode: Acessa o nó pai de um elemento.
  - const pai = elemento.parentNode;

```
const pai = elemento.parentNode;
```

- children: Acessa os filhos de um elemento.
  - const filhos = elemento.children;

```
const filhos = elemento.children;
```

Eventos

Adicionando Event Listeners
Para responder a um evento, você precisa adicionar um "ouvinte de evento" (event listener) a um elemento. Isso é feito usando o método addEventListener().

Aqui estão alguns dos tipos de eventos mais comuns que podem ser usados:

1. Eventos de Mouse:
   - click: Ocorre quando um elemento é clicado.
   - dblclick: Ocorre quando um elemento é clicado duas vezes.
   - mouseover: Ocorre quando o mouse passa sobre um elemento.
   - mouseout: Ocorre quando o mouse sai de um elemento.
   - mousemove: Ocorre quando o mouse é movido sobre um elemento.

```html
<button id="Button">Click</button>
```

```js
const botao = document.getElementById("Button");
botao.addEventListener("click", function() {
    console.log("cliquei no botao")
})
```

Input

Os inputs são elementos interativos — o usuário insere dados e o JavaScript pode ler, alterar ou limpar esses valores em tempo real.

Capturar valores

```
<input type="text" id="nome">
<button id="botao">click</button>
<script>
    var botao = document.getElementById("botao");
    botao.addEventListener("click", function () {
        var nome = document.getElementById("nome").value;
        console.log(nome)
    });
</script>
```

Menu hamburguer

```html
<button id="btn">🍔</button>
<nav>
    <ul id="menu-links" class="menu-links">
        <li>inicio</li>
        <li>serviço</li>
        <li>contato</li>
    </ul>
</nav>
```

```js
btn.addEventListener('click', () => {
    if(menu.style.display == 'block'){
        menu.style.display = 'none';
    }else{
        menu.style.display = 'block';
    }
});
```

```css
#btn {
    display: none;
}

.menu-links {
    display: flex;
    justify-content: center;
    align-items: center;
    gap: 2em;
    list-style-type: none;
}

@media (max-width: 768px) {
    #btn {
        display: block
    }

    .menu-links {
        display: none;
    }
}
```

LocalStorage

O localStorage é um recurso do navegador que permite armazenar dados localmente no computador ou celular do usuário.
Esses dados não são apagados quando a página é recarregada ou o navegador é fechado.

Como funciona

- Cada site tem o seu próprio espaço de armazenamento.
- O conteúdo é salvo como pares de chave e valor (key → value).
- O valor precisa ser uma string, mas você pode converter objetos com JSON.stringify().

`localStorage.setItem(chave, valor)`	Salva um valor
`localStorage.getItem(chave)`	Lê um valor
`localStorage.removeItem(chave)`	Apaga um item específico
`localStorage.clear()`	Limpa tudo
`localStorage.length`	Quantos itens estão salvos

LocalStorage limitações e exemplo

💾 **Tamanho máximo**	Cerca de **5 MB por domínio**
🔓 **Sem segurança**	Qualquer pessoa com acesso ao navegador pode ver o conteúdo
🌐 **Não sincroniza**	É **local**: se o usuário trocar de computador, os dados não vão junto
⏱️ **Sem expiração**	Os dados ficam até serem removidos manualmente

```
let carrinho = JSON.parse(localStorage.getItem('carrinho')) || [];

function adicionarProduto(nome) {
    carrinho.push(nome);
    localStorage.setItem('carrinho', JSON.stringify(carrinho));
}

adicionarProduto('Camiseta');
console.log(carrinho);
```

Formularios

<form>

A **<form>** é uma tag HTML utilizada para coletar dados do usuário. Ela permite que você crie campos de entrada (input fields), botões de envio, caixas de seleção e muito mais, para que os usuários possam enviar informações ao servidor ou manipulá-las no lado do cliente com JavaScript.

```html
<form id="form">
    <div>
        <label for="username">Usuario</label>
        <input type="text" id="nome" placeholder="Digite o seu usuario">
    </div>

    <div>
        <label for="senha">Senha</label>
        <input type="password" id="senha" placeholder="Digite o sua senha">
    </div>

    <button type="submit">Entrar</button>
</form>
```

Capturando Valores com JavaScript

Captura dos Elementos:

- **const nome = document.getElementById("nome")**;: Captura o elemento de entrada de texto com o ID nome e o armazena na variável nome.

- **const senha= document.getElementById("senha")**;: Captura o elemento de entrada de texto com o ID senhae o armazena na variável senha.

- **const form = document.getElementById("form")**;: Captura o formulário com o ID form e o armazena na variável form.

Adicionando um Ouvinte de Evento:

- **form.addEventListener("submit", function(event) { ... })**;: Adiciona um ouvinte de evento ao formulário que escuta o evento de envio (submit).

Impedindo o Envio Padrão:

- **event.preventDefault()**;: Impede o comportamento padrão do formulário, que seria enviar os dados e recarregar a página.

Exibindo os Valores:

- **console.log("Nome:", nome.value)**;: Exibe o valor digitado no campo nome no console.

- **console.log("Senha:", senha.value)**;: Exibe o valor digitado no campo senha no console.

Resumo Final

- Esse código captura os valores dos campos de nome e senha de um formulário, impede o envio padrão do formulário e exibe os valores no console quando o usuário tenta enviar o formulário.

```
1  const nome = document.getElementById("nome");
2  const senha = document.getElementById("senha");
3  const form = document.getElementById("form")
4
5  form.addEventListener("submit", function(event) {
6      // Impede o envio padrão do formulário
7      event.preventDefault();
8
9      // Exibe os valores no console
10     console.log("Nome:", nome.value);
11     console.log("senha:", senha.value);
12 });
```

Comparação: Sincrono x Assíncrono

Sincrono:

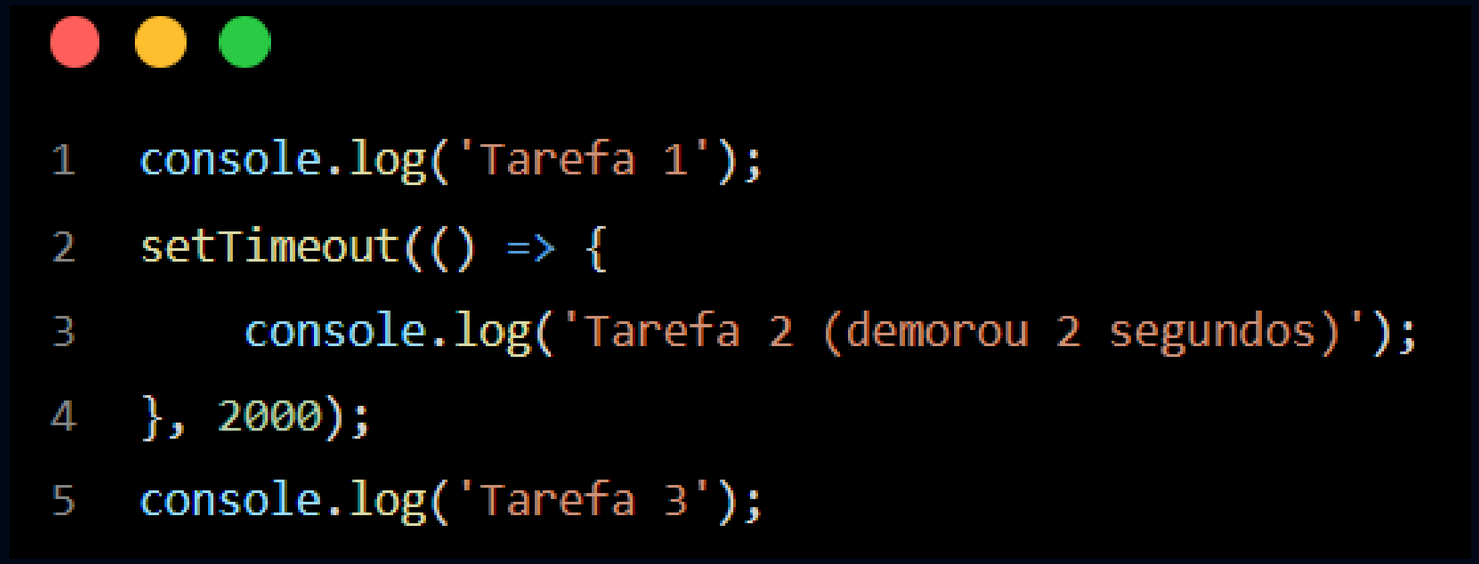
- As tarefas são executadas uma após a outra, em sequência.
- Cada tarefa espera que a anterior seja concluída antes de iniciar a próxima.
- Pode ser problemático quando uma tarefa é lenta, pois o programa todo pode ficar "travado" até que essa tarefa termine.

```
1  console.log('Tarefa 1');
2  // Simulando uma tarefa lenta:
3  for (let i = 0; i < 1000000000; i++) {} // Demora um pouco
4  console.log('Tarefa 2');
```

Comparação: Sincrono x Assíncrono

Assíncrono:

- Permite que algumas tarefas sejam iniciadas e que o código continue executando outras partes enquanto elas não terminam.
- Quando a tarefa assíncrona é concluída, o programa "retoma" e lida com o resultado.
- O código não fica "travado", o que é ideal para operações como requisições de rede ou leitura de arquivos.



```
1  console.log('Tarefa 1');
2  setTimeout(() => {
3      console.log('Tarefa 2 (demorou 2 segundos)');
4  }, 2000);
5  console.log('Tarefa 3');
```

JavaScript e Assíncrono: Promises, async/await

Promises: São objetos que representam o eventual sucesso ou falha de uma operação assíncrona

async/await: Introduzido para simplificar a sintaxe das Promises, permitindo escrever código assíncrono de uma forma mais parecida com código síncrono.

```
async function buscarDados() {
    try {
        const response = await fetch('https://jsonplaceholder.typicode.com/posts');
        const data = await response.json();
        console.log(data);
    } catch (error) {
        console.error('Erro:', error);
    }
}
buscarDados();
```